LAPORAN AKHIR PROYEK

PERBANDINGAN ALGORITMA KLASIFIKASI UNTUK MEMPREDIKSI PELUANG MENDAPATKAN S-RANK PADA SISTEM GACHA ZENLESS ZONE ZERO BERBASIS WEB

Disusun untuk memenuhi Ujian Akhir Semester (UAS)
mata kuliah **Penambangan Data**

Kelas **A11.4502** · Kelompok **6**

NIM	Nama	Peran
A11.2024.16004	Syafiq Yahya	Ketua kelompok, presentasi hasil
A11.2024.15851	Dafi Hauzan A.H	Pengembang aplikasi, pengumpulan dan pengujian data
A11.2024.15842	Gastiadirrijal Rafi M	Penyusun slide presentasi
A11.2024.15826	Zabrina Miftah Z	Penyusun proposal dan laporan
A11.2024.15804	Rayya Hasya Tamimi	Penyusun proposal dan laporan

Dosen Pengampu:
Ardytha Luthfiarta, M.Kom, MCS

PROGRAM STUDI TEKNIK INFORMATIKA – S1
FAKULTAS ILMU KOMPUTER
UNIVERSITAS DIAN NUSWANTORO
SEMARANG

2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa karena atas rahmat-Nya kami dapat menyelesaikan laporan akhir proyek berjudul "Perbandingan algoritma klasifikasi untuk memprediksi peluang mendapatkan S-Rank pada sistem gacha Zenless Zone Zero berbasis web". Dokumen ini disusun untuk memenuhi Ujian Akhir Semester mata kuliah Penambangan Data.

Kami mengucapkan terima kasih kepada Bapak Ardytha Luthfiarta, M.Kom, MCS selaku dosen pengampu yang telah memberikan materi dan arahan, serta kepada teman-teman pemain Zenless Zone Zero yang bersedia menyumbangkan riwayat gacha mereka secara anonim sebagai data penelitian.

Kami menyadari dokumen ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan. Semoga dokumen ini bermanfaat bagi pembaca.

Semarang, Oktober 2026
Kelompok 6

DAFTAR ISI

KATA PENGANTAR.....ii
DAFTAR TABEL.....v
DAFTAR GAMBAR.....vi
BAB I PENDAHULUAN.....1
1.1 Latar Belakang.....1
1.2 Alasan Menggunakan Penambangan Data.....2
1.3 Rumusan Masalah.....2
1.4 Batasan Masalah.....3
1.5 Tujuan.....3
1.6 Manfaat.....3
BAB II TINJAUAN PUSTAKA.....5
2.1 Penambangan Data dan CRISP-DM.....5
2.2 Klasifikasi (Supervised Learning).....5
2.3 Sistem Gacha dan Pity pada Zenless Zone Zero.....5
2.4 Markov Chain.....6
2.5 Algoritma Klasifikasi.....7
2.5.1 Naive Bayes.....7
2.5.2 K-Nearest Neighbor (KNN).....7
2.5.3 Logistic Regression.....8
2.5.4 Decision Tree.....8
2.5.5 Random Forest.....8
2.5.6 XGBoost.....8
2.6 Evaluasi Model.....8
2.7 Pemilihan Hyperparameter dan Uji Signifikansi.....9
2.8 Streamlit.....10
BAB III METODOLOGI.....11
3.1 Alur Penelitian.....11
3.2 Sumber Data.....12
3.3 Pra-pemrosesan Data.....12
3.4 Fitur dan Target.....13
3.5 Pemodelan.....13
3.6 Skenario Evaluasi.....14
3.7 Rancangan Aplikasi.....15
3.8 Alat dan Bahan.....16
BAB IV HASIL DAN PEMBAHASAN.....17
4.1 Implementasi Aplikasi.....17
4.2 Validasi Pengolahan Data.....20

4.3 Hasil Grid Search.....21
4.4 Hasil Tugas A: S-Rank di Pull Berikutnya.....22
4.5 Hasil Tugas B: S-Rank dalam 10 Pull.....23
4.6 Hasil Latih dan Uji dengan Data Asli Saja.....24
4.7 Uji Signifikansi Antar-Model.....27
4.8 Pola Soft Pity pada Data Asli.....28
4.9 Pembahasan.....29
BAB V PENUTUP.....32
5.1 Kesimpulan.....32
5.2 Keterbatasan.....32
5.3 Saran.....33
DAFTAR PUSTAKA.....34
LAMPIRAN.....36

DAFTAR TABEL

Tabel 1.1 Pemeriksaan perlu tidaknya machine learning.....2
Tabel 2.1 Parameter banner terbatas Zenless Zone Zero.....6
Tabel 2.2 Metrik evaluasi yang digunakan.....9
Tabel 3.1 Sumber data penelitian.....12
Tabel 3.2 Fitur yang digunakan.....13
Tabel 3.3 Algoritma, pengaturan tetap, dan ruang pencarian grid search.....14
Tabel 3.4 Halaman aplikasi.....16
Tabel 4.1 Ringkasan data.....21
Tabel 4.2 Hyperparameter hasil grid search pada data simulasi.....21
Tabel 4.3 Hasil cross-validation 5-fold tugas A (rata-rata).....22
Tabel 4.4 Hasil cross-validation 5-fold tugas B (rata-rata) dan uji data asli.....23
Tabel 4.5 Hasil latih–uji dengan data asli saja, tugas B (rata-rata 5-fold, nested grid search).....25
Tabel 4.6 Pengaruh grid search pada eksperimen data asli (accuracy rata-rata 5-fold).....25
Tabel 4.7 Uji t berpasangan terkoreksi tugas B (5-fold).....27
Tabel 4.8 Laju S-Rank per rentang pull: data asli dan model Markov.....28

DAFTAR GAMBAR

Gambar 2.1 Peluang S-Rank per pull (hazard) untuk kedua banner.....7
Gambar 2.2 Ilustrasi rantai Markov sistem pity.....7
Gambar 3.1 Alur penelitian berdasarkan CRISP-DM.....11
Gambar 4.1 Halaman Import Data dengan tiga cara input.....17
Gambar 4.2 Halaman Kalkulator Peluang.....18
Gambar 4.3 Halaman Statistik Riwayat.....19
Gambar 4.4 Halaman Perbandingan Algoritma.....20
Gambar 4.5 Perbandingan akurasi tugas B pada data simulasi dan data asli.....23
Gambar 4.6 Accuracy, Balanced Accuracy, dan F1-score tugas B (cross-validation).....24
Gambar 4.7 Confusion matrix tugas B pada data uji simulasi.....24
Gambar 4.8 Akurasi enam algoritma yang dilatih dan diuji dengan data asli saja 26
Gambar 4.9 Pola soft pity pada data asli dibandingkan model Markov.....28
Gambar 4.10 Kurva peluang mendapat S-Rank dalam 10 pull menurut setiap model.....31
Gambar 4.11 Feature importance model berbasis pohon (tugas B).....31

BAB I
PENDAHULUAN

1.1 Latar Belakang

Gacha adalah mekanisme undian berhadiah di dalam permainan digital. Pemain menukarkan mata uang permainan untuk satu kali undian (*pull*) dan mendapatkan item secara acak sesuai peluang yang ditetapkan pengembang. Mekanisme ini banyak dipakai pada permainan *free-to-play* modern, salah satunya **Zenless Zone Zero** (ZZZ) yang dirilis HoYoverse pada tahun 2024.

Pada ZZZ, hadiah paling langka adalah item **S-Rank**, baik karakter (*Agent*) maupun senjata (*W-Engine*). Peluang dasarnya kecil, yaitu 0,6% untuk Agent dan 1,0% untuk W-Engine pada banner terbatas. Untuk mencegah pemain terlalu lama tidak beruntung, permainan memakai sistem **pity**: semakin lama pemain tidak mendapatkan S-Rank, peluangnya meningkat (*soft pity*) hingga pada batas tertentu S-Rank pasti didapat (*hard pity*, pull ke-90 untuk Agent dan ke-80 untuk W-Engine). Selain itu terdapat aturan 50/50 (Agent) dan 75/25 (W-Engine) yang menentukan apakah S-Rank yang keluar adalah item yang sedang dipromosikan (*rate-up*).

Kombinasi aturan tersebut membuat pemain sulit memperkirakan sendiri peluang mendapatkan S-Rank, misalnya pertanyaan "jika pity saya sekarang 70, berapa peluang saya mendapatkan S-Rank dalam 10 pull berikutnya?". Padahal, riwayat gacha setiap pemain tersimpan di server dan dapat diambil sebagai data. Data riwayat ini berpotensi diolah dengan teknik penambangan data untuk menjawab pertanyaan tersebut secara kuantitatif.

Penambangan data menyediakan banyak algoritma klasifikasi, di antaranya Naive Bayes, K-Nearest Neighbor (KNN), Logistic Regression, Decision Tree, Random Forest, dan XGBoost. Masing-masing memiliki asumsi dan cara belajar yang berbeda, sehingga kinerjanya perlu dibandingkan pada kasus yang sama. Kasus gacha juga memiliki tantangan khusus, yaitu **data yang sangat tidak seimbang**: kelas "mendapat S-Rank" hanya sekitar 1,7% dari seluruh pull, sehingga metrik akurasi saja dapat menyesatkan.

Berdasarkan hal tersebut, proyek ini membangun aplikasi berbasis web yang dapat diakses dari ponsel maupun komputer untuk menghitung peluang mendapatkan S-Rank dengan model Markov Chain, sekaligus membandingkan kinerja enam algoritma klasifikasi dalam memprediksi keluarnya S-Rank.

1.2 Alasan Menggunakan Penambangan Data

Sebelum memilih algoritma, perlu dipastikan bahwa kasus ini memang membutuhkan penambangan data. *Machine learning* tidak diperlukan apabila (1) hasil dapat ditentukan langsung oleh suatu atribut, atau (2) sudah tersedia pengetahuan atau rumus yang pasti dan lengkap. Tabel 1.1 menunjukkan bahwa kedua kondisi tersebut tidak terpenuhi pada sistem gacha Zenless Zone Zero.

Tabel 1.1 Pemeriksaan perlu tidaknya machine learning

Kondisi	**Pada gacha Zenless Zone Zero**	**Terpenuhi?**
Ada atribut yang langsung menentukan hasil	Tidak ada. Pity hanya mengubah peluang; hasil setiap pull tetap acak. Kepastian hanya terjadi di hard pity.	Tidak
Ada rumus yang pasti dan lengkap	Hanya sebagian. Rate dasar, rate gabungan, hard pity, dan peluang rate-up diumumkan resmi, tetapi kapan soft pity dimulai dan seberapa cepat peluang naik tidak diumumkan. Perkiraan komunitas pun berbeda-beda.	Tidak

Bagian aturan yang tidak diumumkan tersebut hanya dapat diketahui dari data riwayat gacha pemain. Karena setiap pull memiliki label yang jelas (mendapat S-Rank atau tidak), masalah ini termasuk **klasifikasi** sekaligus **estimasi** peluang dalam *supervised learning*. Model Markov Chain tetap dipakai, tetapi sebagai **pembanding teori** yang dibangun dari parameter resmi ditambah estimasi soft pity. Algoritma klasifikasi tidak diberi tahu tentang soft pity, sehingga dapat diuji apakah mampu menemukan pola tersebut sendiri dari data.

1.3 Rumusan Masalah

1. Bagaimana menghitung peluang pemain mendapatkan S-Rank dan S-Rank *rate-up* dalam N pull berdasarkan pity dan status *guaranteed* pada banner terbatas Zenless Zone Zero?
2. Apakah pola kenaikan peluang (*soft pity*) yang tidak diumumkan resmi dapat ditemukan dari data riwayat gacha pemain?
3. Algoritma klasifikasi manakah di antara Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, dan XGBoost yang paling akurat memprediksi keluarnya S-Rank?
4. Bagaimana mengevaluasi model secara tepat pada data yang sangat tidak seimbang?

5. Bagaimana menyajikan hasil tersebut dalam aplikasi web yang mudah diakses pengguna ponsel maupun komputer?

1.4 Batasan Masalah

- Banner yang diteliti hanya **banner Agent terbatas** (*Exclusive Channel*) dan **banner W-Engine terbatas** (*W-Engine Channel*). Banner standar dan Bangboo tidak dibahas.
- Yang diprediksi adalah keluarnya item **S-Rank**; item A-Rank dan B-Rank tidak dimodelkan.
- Parameter resmi (rate dasar, rate gabungan, hard pity, peluang *rate-up*) diambil dari detail banner di dalam permainan. Titik mulai *soft pity* tidak diumumkan resmi, sehingga model Markov memakai estimasi komunitas yang kemudian diperiksa dengan data asli.
- Data asli berasal dari riwayat akun anggota kelompok dan teman yang bersedia, dalam bentuk anonim.
- Aplikasi dibangun dengan Python dan Streamlit, dan pengambilan riwayat otomatis hanya tersedia untuk permainan versi PC (Windows).

1.5 Tujuan

1. Membangun model Markov Chain untuk menghitung peluang mendapatkan S-Rank dan S-Rank *rate-up* berdasarkan parameter banner.
2. Menggali pola *soft pity* yang tidak diumumkan resmi dari data riwayat gacha pemain.
3. Membandingkan kinerja enam algoritma klasifikasi dalam memprediksi keluarnya S-Rank, dengan hyperparameter yang dipilih melalui *grid search* dan perbedaan kinerja yang diuji signifikansinya secara statistik.
4. Menerapkan metrik evaluasi yang sesuai untuk data tidak seimbang, yaitu Accuracy, Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC, Log Loss, dan Brier Score.
5. Menyajikan seluruh hasil dalam aplikasi web yang dapat diakses dari ponsel, tablet, maupun komputer.

1.6 Manfaat

- **Bagi pemain:** mendapat gambaran peluang yang jelas untuk merencanakan penggunaan mata uang permainan.

- **Bagi mahasiswa:** menerapkan tahapan penambangan data (CRISP-DM), algoritma klasifikasi, evaluasi dengan *confusion matrix*, dan *deployment* dengan Streamlit sesuai materi perkuliahan.
- **Bagi keilmuan:** memberikan contoh penerapan penambangan data untuk menggali aturan yang tidak diumumkan (soft pity) dari data, sekaligus contoh nyata bahaya memakai akurasi sebagai satu-satunya metrik pada data tidak seimbang.

BAB II
TINJAUAN PUSTAKA

2.1 Penambangan Data dan CRISP-DM

Penambangan data (*data mining*) adalah proses menemukan pola, pengetahuan, dan informasi yang berguna dari kumpulan data berukuran besar (Han, Kamber, & Pei, 2012). Proses ini mengubah **data** mentah menjadi **informasi**, lalu menjadi **pengetahuan** yang dapat dipakai untuk mengambil keputusan.

Proyek ini mengikuti kerangka **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*) yang terdiri atas enam tahap: *business understanding*, *data understanding*, *data preparation*, *modeling*, *evaluation*, dan *deployment* (Chapman et al., 2000).

2.2 Klasifikasi (Supervised Learning)

Klasifikasi adalah tugas *supervised learning* untuk memetakan sekumpulan fitur ke salah satu label kelas berdasarkan contoh data berlabel. Pada proyek ini, label berupa dua kelas (klasifikasi biner): **mendapat S-Rank** atau **tidak mendapat S-Rank**. Selain label, sebagian besar algoritma juga menghasilkan **peluang** masuk ke tiap kelas. Peluang inilah yang membuat klasifikasi juga berperan sebagai **estimasi**, karena pemain tidak hanya membutuhkan tebakan ya atau tidak, tetapi juga seberapa besar peluangnya.

2.3 Sistem Gacha dan Pity pada Zenless Zone Zero

Fitur gacha pada Zenless Zone Zero disebut *Signal Search*. Setiap pull memiliki peluang tertentu menghasilkan item S-Rank. Parameter banner terbatas yang dipakai dalam proyek ini ditunjukkan pada Tabel 2.1. Angka resmi diambil dari menu detail *Signal Search* di dalam permainan (HoYoverse, 2024).

Tabel 2.1 Parameter banner terbatas Zenless Zone Zero

Parameter	Agent Terbatas	W-Engine Terbatas	Sumber
Rate dasar S-Rank	0,6%	1,0%	Resmi (detail banner)
Rate gabungan (termasuk pity)	1,6%	2,0%	Resmi (detail banner)
Hard pity	90	80	Resmi (detail banner)
Peluang S-Rank rate-up	50%	75%	Resmi (detail banner)
Soft pity mulai	pull ke-74	pull ke-65	Estimasi komunitas (tidak resmi)

Jika pemain mendapatkan S-Rank yang bukan *rate-up* (disebut kalah 50/50), maka S-Rank berikutnya **pasti** *rate-up*. Status ini disebut *guaranteed*. Riwayat gacha dapat diambil dari server resmi melalui URL yang memuat kunci akses sementara (*authkey*), dan dapat dipertukarkan antaraplikasi dengan format standar **UIGF** (*Uniformed Interchangeable GachaLog Format*) versi 4 (UIGF Organization, 2024).

2.4 Markov Chain

Markov Chain adalah proses stokastik ketika peluang berpindah ke keadaan (*state*) berikutnya hanya bergantung pada keadaan saat ini (Norris, 1997). Sistem pity cocok dimodelkan dengan Markov Chain karena peluang mendapatkan S-Rank pada suatu pull hanya bergantung pada jumlah pull sejak S-Rank terakhir (pity) dan status *guaranteed*.

Peluang mendapatkan S-Rank pada pull ke-n, dengan syarat belum mendapat S-Rank sebelumnya, disebut *hazard* h(n):

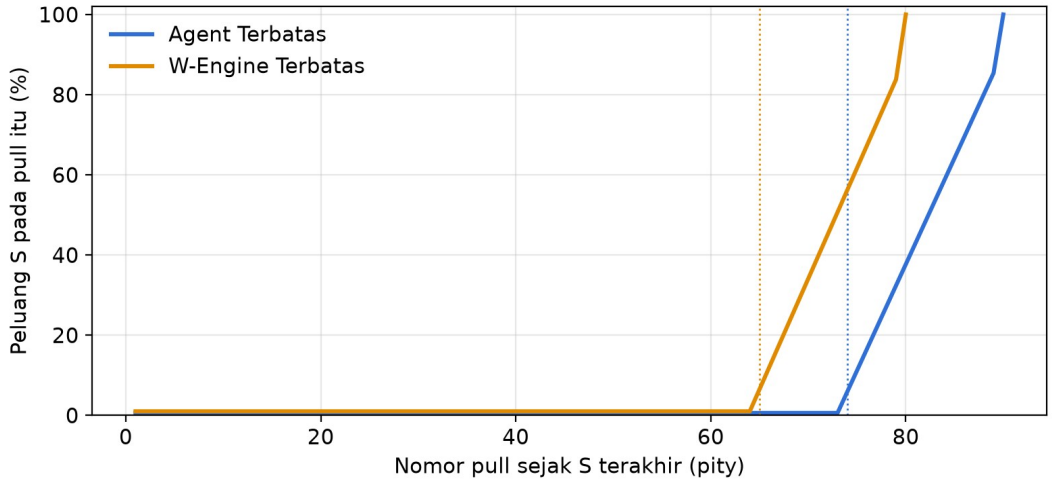
$$h(n) = p \qquad \text{untuk } n < s$$

$$h(n) = \min(1,\ p + k\cdot(n - s + 1)) \quad \text{untuk } n \geq s$$

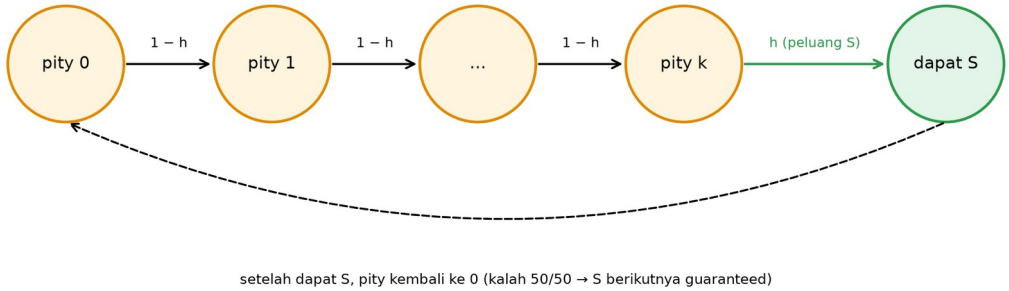
$$h(\text{hard pity}) = 1$$

dengan p = rate dasar, s = titik mulai soft pity, dan k = kenaikan peluang per pull. Karena nilai k tidak diumumkan, k dicari dengan **metode biseksi** sehingga rata-rata jumlah pull per S-Rank sama dengan kebalikan rate gabungan resmi (62,5 pull untuk Agent dan 50 pull untuk W-Engine). Hasilnya k ≈ 5,30% untuk Agent

dan k ≈ 5,52% untuk W-Engine. Nilai s (pull ke-74 dan ke-65) juga bukan angka resmi, melainkan estimasi komunitas yang diperiksa dengan data asli dalam penelitian ini.



Gambar 2.1 Peluang S-Rank per pull (hazard) untuk kedua banner



Gambar 2.2 Ilustrasi rantai Markov sistem pity

2.5 Algoritma Klasifikasi

2.5.1 Naive Bayes

Naive Bayes adalah pengklasifikasi probabilistik berdasarkan Teorema Bayes dengan asumsi bahwa setiap fitur saling bebas jika kelasnya diketahui. Varian Gaussian Naive Bayes mengasumsikan setiap fitur numerik berdistribusi normal di dalam setiap kelas. Algoritma ini sangat cepat, namun asumsinya sering tidak terpenuhi pada data nyata.

2.5.2 K-Nearest Neighbor (KNN)

KNN mengklasifikasikan data baru berdasarkan mayoritas label dari k data latih terdekat menurut jarak tertentu, umumnya jarak Euclidean (Cover & Hart,

1967). KNN tidak membangun model eksplisit (*lazy learner*), sehingga fitur perlu dinormalisasi agar tidak ada fitur yang mendominasi perhitungan jarak.

2.5.3 Logistic Regression

Logistic Regression memodelkan peluang kelas positif dengan fungsi sigmoid dari kombinasi linear fitur: $P(y = 1) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)})$ (Hosmer, Lemeshow, & Sturdivant, 2013). Model ini mudah ditafsirkan, tetapi hanya mampu membentuk batas keputusan yang linear terhadap fitur.

2.5.4 Decision Tree

Decision Tree membagi data secara bertahap berdasarkan nilai fitur sehingga membentuk struktur pohon aturan *jika–maka* (Quinlan, 1986). Pembagian dipilih untuk memaksimalkan kemurnian kelas, misalnya dengan indeks Gini. Kelebihannya, aturan yang dihasilkan mudah dibaca manusia.

2.5.5 Random Forest

Random Forest adalah gabungan (*ensemble*) banyak Decision Tree yang masing-masing dilatih pada sampel acak data dan subset fitur acak, lalu hasilnya dirata-ratakan (*bagging*) (Breiman, 2001). Teknik ini mengurangi *overfitting* dibandingkan satu pohon.

2.5.6 XGBoost

XGBoost (*Extreme Gradient Boosting*) membangun pohon secara berurutan, di mana setiap pohon baru memperbaiki kesalahan pohon sebelumnya dengan optimasi gradien dan regularisasi (Chen & Guestrin, 2016). Algoritma ini dikenal kuat pada data tabular.

2.6 Evaluasi Model

Evaluasi klasifikasi biner didasarkan pada **confusion matrix** yang berisi True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Metrik yang dipakai dalam proyek ini ditunjukkan pada Tabel 2.2.

Tabel 2.2 Metrik evaluasi yang digunakan

Metrik	**Rumus / pengertian**
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1-score	2 × Precision × Recall / (Precision + Recall)
Balanced Accuracy	rata-rata recall kelas positif dan kelas negatif
ROC-AUC	luas di bawah kurva ROC; kemampuan membedakan kelas di semua ambang
Log Loss	−rata-rata [y·log(p) + (1−y)·log(1−p)]; menilai ketepatan angka peluang
Brier Score	rata-rata $(p - y)^2$; makin kecil, makin tepat angka peluangnya

Pada data tidak seimbang, akurasi dapat tinggi walaupun model tidak pernah menebak kelas minoritas (He & Garcia, 2009). Karena itu metrik seperti Balanced Accuracy, F1-score, dan ROC-AUC perlu digunakan sebagai pendamping (Saito & Rehmsmeier, 2015). Log Loss dan Brier Score (Brier, 1950) dipakai untuk menilai seberapa tepat peluang yang dihasilkan.

2.7 Pemilihan Hyperparameter dan Uji Signifikansi

Hyperparameter adalah pengaturan algoritma yang tidak dipelajari dari data, misalnya jumlah tetangga k pada KNN atau kedalaman maksimum pohon. *Grid search* mencoba setiap kombinasi nilai dari ruang pencarian yang ditentukan, lalu memilih kombinasi dengan skor cross-validation terbaik (Pedregosa et al., 2011). Data yang dipakai untuk memilih hyperparameter tidak boleh sama dengan data yang dipakai untuk menilai model akhir, karena skornya akan terlihat lebih baik daripada kenyataan. Jika data terbatas, dipakai *nested cross-validation*: grid search dijalankan hanya di dalam data latih setiap fold, sedangkan data uji fold tersebut hanya dipakai untuk penilaian (Varma & Simon, 2006).

Selisih akurasi antar-model juga perlu diuji, apakah bermakna secara statistik atau hanya kebetulan. Uji t berpasangan biasa pada hasil setiap fold terlalu mudah menyatakan dua model berbeda, karena data latih antarfold saling tumpang tindih sehingga hasilnya tidak saling bebas. Nadeau dan Bengio (2003) mengusulkan **uji t berpasangan terkoreksi** (*corrected resampled t-test*) yang memperbesar varians dengan faktor koreksi:

$$t = \bar{d} \,/\, \sqrt{\left(1/k + n_{uji} / n_{latih}\right) \cdot s^2_d}$$

dengan $\bar{d}$ dan s^2_d adalah rata-rata dan varians selisih akurasi dua model pada k fold, sedangkan n_uji/n_latih adalah perbandingan ukuran data uji dan data latih. Nilai t dibandingkan dengan distribusi t berderajat bebas $k - 1$. Jika p-value kurang dari 0,05, selisih kedua model dinyatakan **signifikan**.

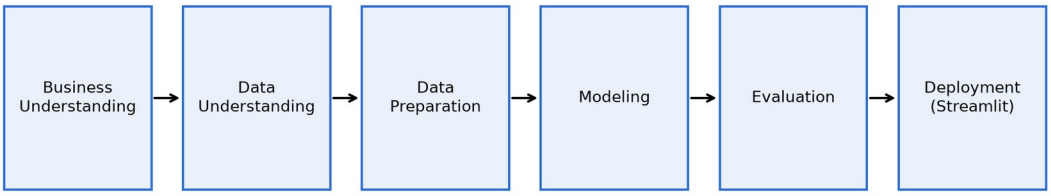
2.8 Streamlit

Streamlit adalah pustaka Python sumber terbuka untuk membangun aplikasi web interaktif bagi analisis data dan *machine learning* tanpa perlu menulis HTML, CSS, atau JavaScript (Streamlit Inc., 2024). Aplikasi Streamlit dapat dipublikasikan gratis melalui Streamlit Community Cloud sehingga dapat dibuka dari peramban ponsel maupun komputer.

BAB III
METODOLOGI

3.1 Alur Penelitian

Penelitian dilaksanakan mengikuti tahapan CRISP-DM seperti pada Gambar 3.1.



Gambar 3.1 Alur penelitian berdasarkan CRISP-DM

1. **Business understanding:** merumuskan kebutuhan pemain, yaitu mengetahui peluang mendapatkan S-Rank dari pity saat ini.
2. **Data understanding:** mempelajari struktur riwayat gacha (id, jenis banner, nama item, rank, waktu), aturan banner, dan pola laju S-Rank per pity pada data asli.
3. **Data preparation:** membersihkan data, menghitung pity dan status guaranteed, membuang data yang tidak lengkap, dan membentuk fitur.
4. **Modeling:** memilih hyperparameter dengan grid search, melatih enam algoritma klasifikasi, dan menyusun model Markov Chain sebagai acuan teori.
5. **Evaluation:** membandingkan model dengan cross-validation dan uji signifikansi, lalu menguji pada data pemain asli.
6. **Deployment:** menerapkan model ke aplikasi web Streamlit.

3.2 Sumber Data

Tabel 3.1 Sumber data penelitian

Sumber	Keterangan	Kegunaan
Simulasi Monte Carlo	400 akun virtual, 80–700 pull per banner, dibangkitkan dari model hazard (seed 42); ±314 ribu pull	Data latih dan uji utama
Simulasi untuk grid search	100 akun virtual tambahan (seed 43); ±77 ribu baris	Pemilihan hyperparameter, tidak dipakai untuk evaluasi
Data asli pemain	3 akun (anggota kelompok dan teman), 1.751 pull, 28 S-Rank; anonim tanpa UID, nama item, dan waktu	Uji model simulasi, dan eksperimen latih–uji dengan data asli saja

Simulasi dipakai karena riwayat satu akun hanya berisi ratusan pull dengan sedikit S-Rank, sehingga tidak cukup untuk melatih model. Namun, simulasi dibangkitkan dari model Markov yang memuat estimasi soft pity, sehingga model yang dilatih dengan simulasi pada dasarnya mempelajari ulang asumsi tersebut. Karena itu data asli memegang peran penting: menguji apakah model dari simulasi berlaku untuk pemain sungguhan, sekaligus memeriksa pola soft pity langsung dari data.

Selain itu, sebagai eksperimen kedua, keenam algoritma juga dilatih dan diuji **hanya dengan data asli** (1.425 baris setelah pembersihan). Eksperimen ini memastikan bahwa hasil perbandingan tidak bergantung pada data simulasi, dan jumlah barisnya sudah memenuhi ketentuan dataset tabel minimal 500–1.000 baris.

Riwayat asli diambil dengan tiga cara yang disediakan aplikasi: (1) skrip PowerShell yang membaca URL riwayat dari *cache* permainan di PC, (2) unggah berkas UIGF, dan (3) input manual. Skrip hanya membaca berkas di komputer pemain dan tidak mengirim data ke mana pun; *authkey* hanya dipakai sekali dan tidak disimpan.

3.3 Pra-pemrosesan Data

- Hanya pull dari banner Agent dan W-Engine terbatas yang diambil.
- Data duplikat berdasarkan kolom id dibuang, lalu data diurutkan kronologis per akun dan per banner.

- Pity dan status guaranteed dihitung ulang dari urutan pull. Kalah 50/50 ditentukan dari daftar S-Rank standar.
- **Segmen pertama** (pull sebelum S-Rank pertama yang tercatat) dibuang, karena server hanya menyimpan riwayat beberapa bulan terakhir sehingga pity pada segmen ini bisa tidak lengkap.
- Riwayat yang melebihi hard pity tanpa S-Rank ditolak karena tidak masuk akal.

3.4 Fitur dan Target

Setiap baris data adalah satu titik keputusan **sebelum** sebuah pull, sehingga tidak ada informasi masa depan yang bocor ke model.

Tabel 3.2 Fitur yang digunakan

Fitur	Keterangan
pity	jumlah pull sejak S-Rank terakhir (0 sampai hard pity − 1)
pity_ratio	pity dibagi hard pity, agar dua banner sebanding
guaranteed	1 jika S-Rank berikutnya pasti rate-up, 0 jika tidak
is_wengine	1 untuk banner W-Engine, 0 untuk banner Agent

Dua tugas klasifikasi biner didefinisikan:

- **Tugas A:** apakah pull berikutnya menghasilkan S-Rank? (kelas positif ±1,7%)
- **Tugas B:** apakah pemain mendapat minimal satu S-Rank dalam 10 pull berikutnya (satu kali *ten-pull*)? (kelas positif ±16,6%)

3.5 Pemodelan

Setiap algoritma memiliki pengaturan tetap dan beberapa *hyperparameter* yang dipilih dengan *grid search* (Tabel 3.3). Nilai pertama pada setiap ruang pencarian adalah pengaturan awal sebelum grid search, sehingga jika beberapa kombinasi memperoleh skor yang sama persis, pengaturan awal dipertahankan.

Tabel 3.3 Algoritma, pengaturan tetap, dan ruang pencarian grid search

Algoritma	Pengaturan tetap	Ruang pencarian grid search
Naive Bayes	Gaussian Naive Bayes	var_smoothing ∈ {10^{-9}; 10^{-6}; 10^{-3}; 0,01; 0,1; 1}
KNN	normalisasi StandardScaler, jarak Euclidean	k ∈ {101; 5; 15; 31; 51; 201}
Logistic Regression	normalisasi StandardScaler, max_iter 2000	C ∈ {1; 0,01; 0,1; 10}
Decision Tree	indeks Gini	max_depth ∈ {6; 3; 4; 8}; min_samples_leaf ∈ {50; 20; 100}
Random Forest	150 pohon	max_depth ∈ {8; 6; 12}; min_samples_leaf ∈ {20; 10; 50}
XGBoost	300 pohon, subsample 0,8	max_depth ∈ {4; 3; 6}; learning_rate ∈ {0,05; 0,1}
Pembanding: Baseline	selalu menebak "tidak dapat S-Rank" (DummyClassifier)	–
Pembanding: Markov Chain	peluang teoretis dari model hazard (Subbab 2.4); pada data simulasi menjadi batas atas akurasi karena data dibangkitkan dari model yang sama	–

3.6 Skenario Evaluasi

- Data dibagi **per akun**, bukan per pull, karena pull berurutan dari akun yang sama saling berkaitan. Pembagian per pull akan membocorkan informasi antara data latih dan data uji.
- **Grid search** pada data simulasi: hyperparameter dipilih memakai 100 akun simulasi tambahan (seed 43) yang tidak dipakai untuk evaluasi, dengan GroupKFold 4-fold per akun dan metrik accuracy. Pengaturan terpilih lalu dipakai untuk semua evaluasi simulasi dan model di aplikasi.
- **GroupKFold 5-fold cross-validation** dipakai untuk membandingkan model secara stabil (rata-rata ± simpangan baku).
- **Uji signifikansi:** akurasi setiap algoritma pada kelima fold dibandingkan dengan algoritma berakurasi tertinggi memakai uji t berpasangan terkoreksi (Subbab 2.7, $\alpha = 0{,}05$).
- **GroupShuffleSplit 80/20** dipakai untuk model akhir yang disimpan ke aplikasi.

- Model akhir diuji ulang pada **data asli 3 akun** yang tidak pernah dipakai saat pelatihan.
- **Eksperimen kedua:** keenam algoritma dilatih dan diuji hanya dengan data asli (1.425 baris) memakai GroupKFold 5-fold yang dibagi per **siklus pity** (25 siklus). Pembagian per siklus dipakai karena data asli hanya berasal dari 3 akun, dan siklus pity saling bebas karena peluang selalu kembali ke awal setelah mendapat S-Rank. Hyperparameter pada eksperimen ini dipilih dengan *nested cross-validation*: grid search dijalankan ulang di dalam data latih setiap fold (GroupKFold 4-fold per siklus), sehingga data uji tidak pernah ikut memilih pengaturan.
- **Analisis pola soft pity:** laju S-Rank per rentang pull pada data asli dibandingkan dengan peluang model Markov, dan aturan Decision Tree yang dilatih dengan data asli diperiksa untuk melihat di pity berapa model memisahkan data.
- Prediksi kelas memakai ambang peluang 0,5. *Oversampling* (misalnya SMOTE) tidak dipakai karena tujuan utamanya adalah angka peluang yang tepat, sedangkan menambah data buatan untuk kelas minoritas akan menggeser peluang tersebut. Ketidakseimbangan kelas ditangani dengan memilih metrik yang sesuai.
- Metrik: Accuracy (metrik utama), Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC, Log Loss, dan Brier Score.

3.7 Rancangan Aplikasi

Aplikasi web dibangun dengan Streamlit, menggunakan tema gelap, dan dapat diakses dari ponsel maupun komputer. Halaman yang tersedia ditunjukkan pada Tabel 3.4.

Tabel 3.4 Halaman aplikasi

Halaman	Fungsi
Beranda	penjelasan aplikasi dan aturan banner
Import Data	tiga cara input: URL dari PowerShell, unggah UIGF, input manual; tersedia data contoh
Kalkulator Peluang	peluang S-Rank dan S-Rank rate-up dalam N pull (Markov Chain) dan prediksi 6 algoritma
Statistik Riwayat	pity setiap S-Rank, rata-rata pity, menang/kalah 50/50, pity saat ini
Perbandingan Algoritma	tabel metrik, cross-validation, uji signifikansi, hasil grid search, latih–uji data asli, pola soft pity, kurva peluang, confusion matrix, feature importance
Panduan & FAQ	cara menjalankan skrip PowerShell dan tanya jawab

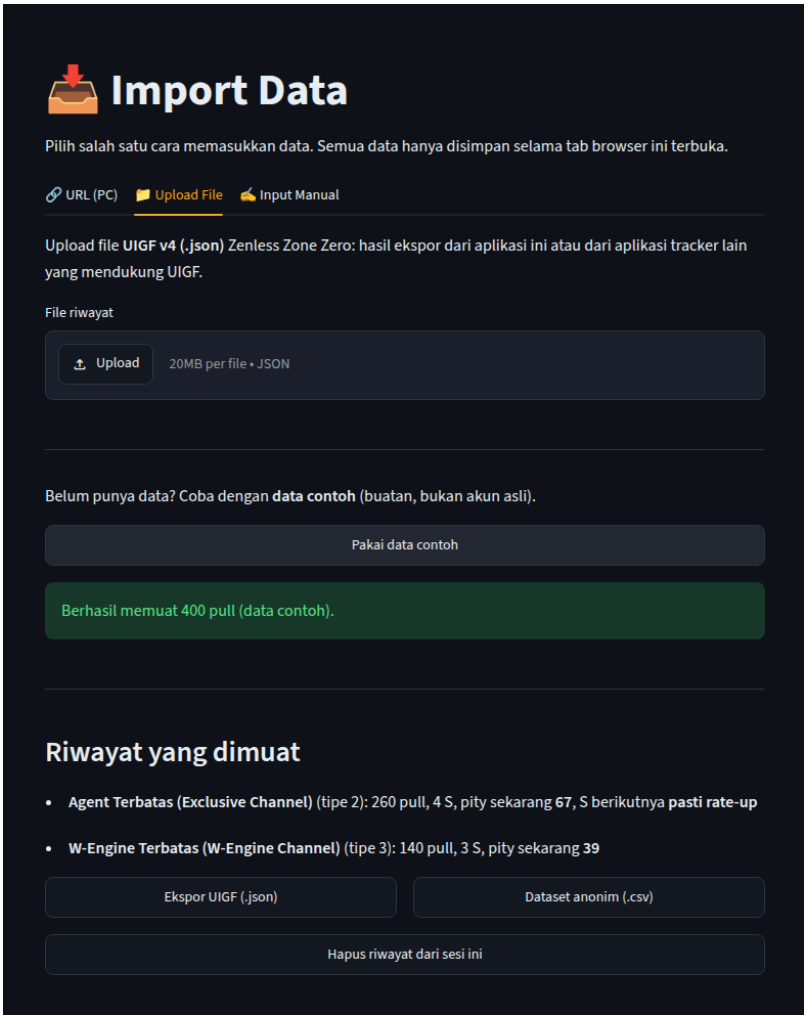
3.8 Alat dan Bahan

- Bahasa pemrograman Python 3.12 dengan pustaka pandas, NumPy, SciPy, scikit-learn 1.9, XGBoost 3.2, dan Plotly.
- Streamlit 1.64 dan Streamlit Community Cloud untuk *deployment*.
- Pytest dan GitHub Actions untuk pengujian otomatis (106 kasus uji).
- Windows PowerShell untuk skrip pengambil URL riwayat.
- GitHub untuk penyimpanan kode sumber.

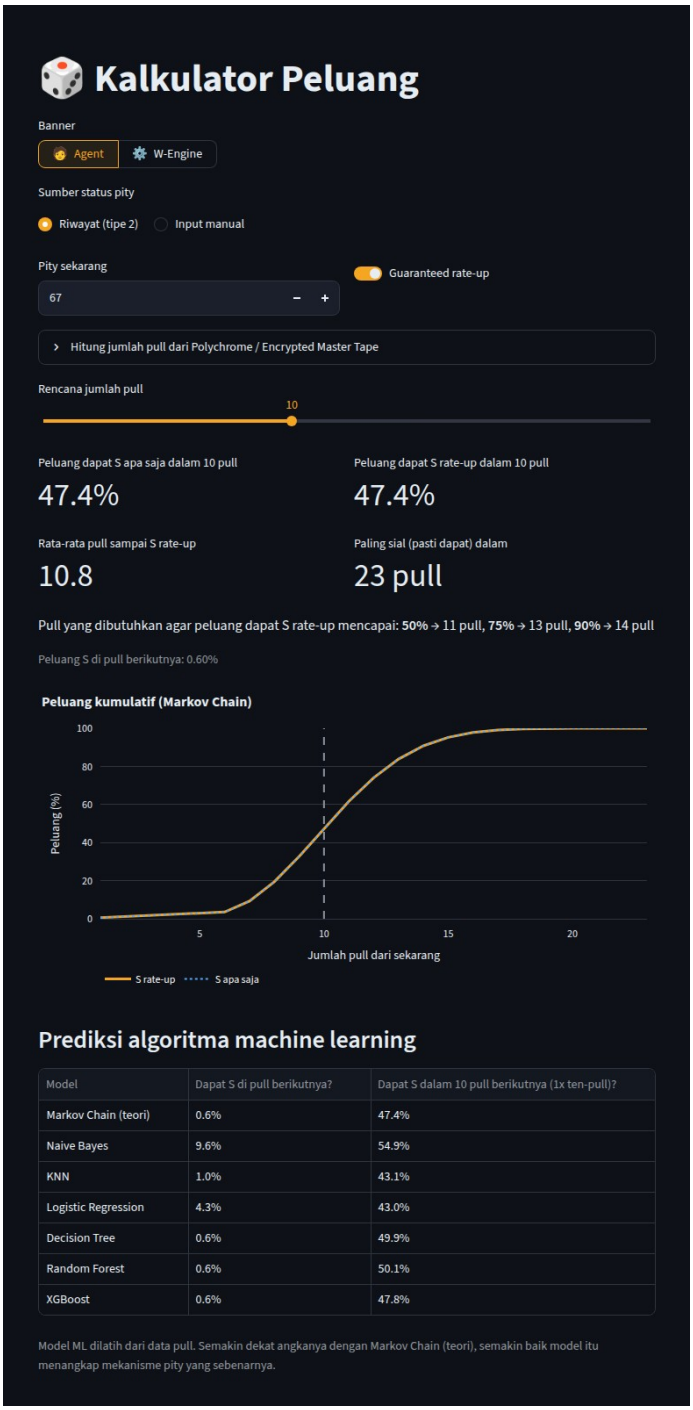
BAB IV
HASIL DAN PEMBAHASAN

4.1 Implementasi Aplikasi

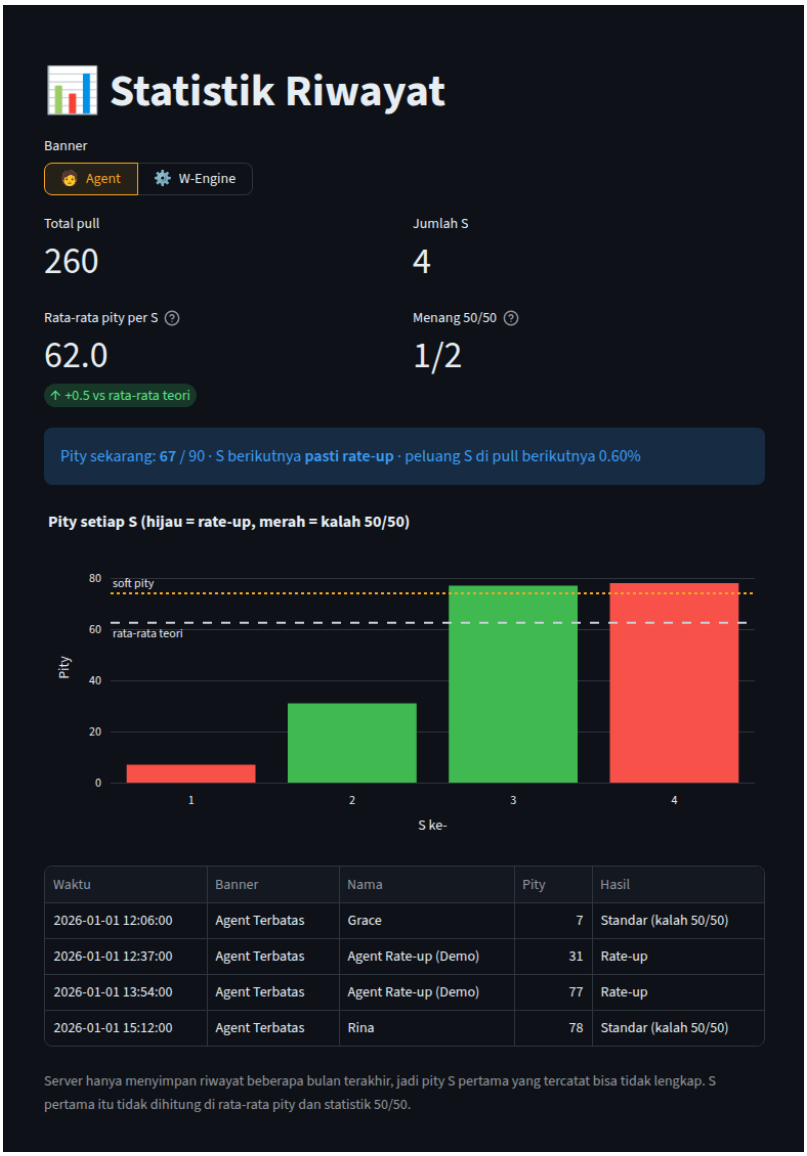
Aplikasi berhasil dipublikasikan dan dapat diakses di **https://project-data-mining-zzz.streamlit.app**. Kode sumber tersedia di https://github.com/tehgeii/project-data-mining. Gambar 4.1 sampai Gambar 4.4 menampilkan beberapa halaman aplikasi.



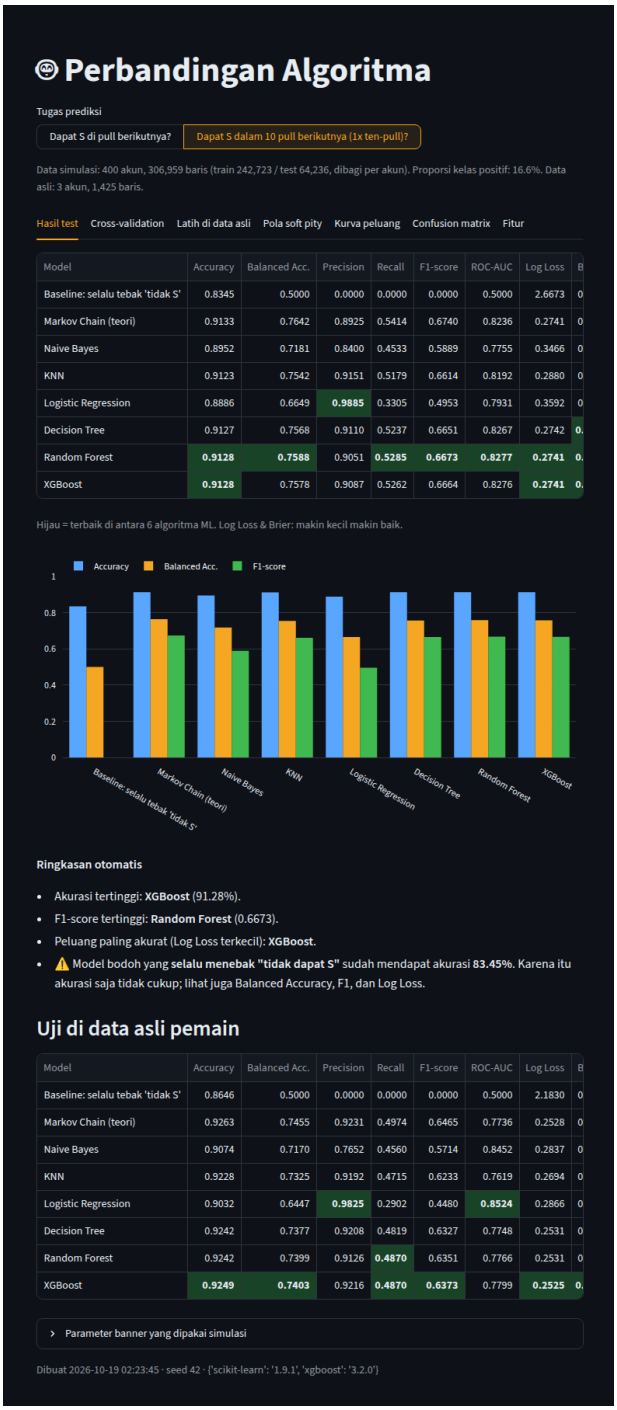
Gambar 4.1 Halaman Import Data dengan tiga cara input



Gambar 4.2 Halaman Kalkulator Peluang



Gambar 4.3 Halaman Statistik Riwayat



Gambar 4.4 Halaman Perbandingan Algoritma

4.2 Validasi Pengolahan Data

Riwayat tiga akun asli diambil melalui skrip PowerShell. Pada saat pengecekan, pity yang dihitung aplikasi untuk akun pertama (permainan versi Steam), yaitu 70/90 pada banner Agent dan 46/80 dengan status guaranteed pada banner W-Engine, **sama persis** dengan yang tampil di dalam permainan. Hal ini

menunjukkan bahwa pengambilan data, perhitungan pity, dan deteksi 50/50 sudah benar. Ringkasan data yang dipakai ditunjukkan pada Tabel 4.1.

Tabel 4.1 Ringkasan data

Keterangan	Simulasi	Data asli
Jumlah akun	400	3
Baris tugas A (pull berikutnya)	314.159	1.479
Baris tugas B (10 pull)	306.959	1.425
Proporsi kelas positif tugas A	1,7%	1,5%
Proporsi kelas positif tugas B	16,6%	13,5%
Data latih / uji (tugas B)	242.723 / 64.236	– / 1.425
Data grid search (tugas B)	100 akun tambahan, 77.156 baris	dipilih ulang di setiap fold
Eksperimen latih–uji data asli (tugas B)	–	1.425 baris, 25 siklus, 5-fold

4.3 Hasil Grid Search

Grid search dijalankan pada 100 akun simulasi tambahan (78.956 baris tugas A dan 77.156 baris tugas B) yang tidak dipakai untuk evaluasi. Hyperparameter yang terpilih ditunjukkan pada Tabel 4.2.

Tabel 4.2 Hyperparameter hasil grid search pada data simulasi

Algoritma	Terpilih untuk tugas A	Terpilih untuk tugas B	Accuracy tugas B: awal → terpilih
Naive Bayes	var_smoothing = 10^{-9}	var_smoothing = 10^{-3}	0,8543 → **0,8897**
KNN	k = 101	k = 51	0,9069 → 0,9070
Logistic Regression	C = 1	C = 10	0,8843 → 0,8853
Decision Tree	max_depth = 4, min_samples_leaf = 20	max_depth = 6, min_samples_leaf = 100	0,9068 → 0,9070
Random Forest	max_depth = 6, min_samples_leaf = 10	max_depth = 8, min_samples_leaf = 50	0,9068 → 0,9070
XGBoost	learning_rate = 0,05, max_depth = 3	learning_rate = 0,05, max_depth = 3	0,9068 → 0,9069

- **Naive Bayes paling diuntungkan.** Pada tugas B, var_smoothing = 10^{-3} menaikkan akurasi grid search dari 0,8543 menjadi 0,8897. Nilai ini

menambah varians setiap fitur sehingga peluang yang dihasilkan tidak terlalu ekstrem.

- **Algoritma lain hampir tidak berubah** (kenaikan paling besar 0,1 poin). Pengaturan awal KNN, Logistic Regression, Decision Tree, Random Forest, dan XGBoost sudah mendekati yang terbaik untuk data simulasi yang besar.
- Pada tugas A, perbedaan akurasi antarpengaturan hanya terlihat pada desimal keempat (±98,3%), sehingga pemilihan hyperparameter hampir tidak berpengaruh.
- Pengaturan terpilih dipakai untuk semua hasil simulasi pada Subbab 4.4 dan 4.5 serta untuk model di aplikasi. Pada eksperimen data asli (Subbab 4.6), hyperparameter dipilih ulang di dalam setiap fold.

4.4 Hasil Tugas A: S-Rank di Pull Berikutnya

Tabel 4.3 Hasil cross-validation 5-fold tugas A (rata-rata)

Model	Accuracy	Bal. Acc.	F1	ROC-AUC	Log Loss
Baseline (selalu "tidak S")	0,9828	0,5000	0,0000	0,5000	0,2768
Markov Chain (teori)	0,9830	0,5184	0,0699	0,7890	0,0662
Naive Bayes	0,9829	0,5007	0,0030	0,7563	0,0792
KNN	0,9829	0,5150	0,0574	0,7728	0,0985
Logistic Regression	0,9828	0,5000	0,0000	0,7578	0,0776
Decision Tree	0,9830	0,5128	0,0498	0,7888	0,0662
Random Forest	0,9829	0,5141	0,0543	0,7890	0,0663
XGBoost	0,9830	0,5138	0,0533	0,7878	0,0663

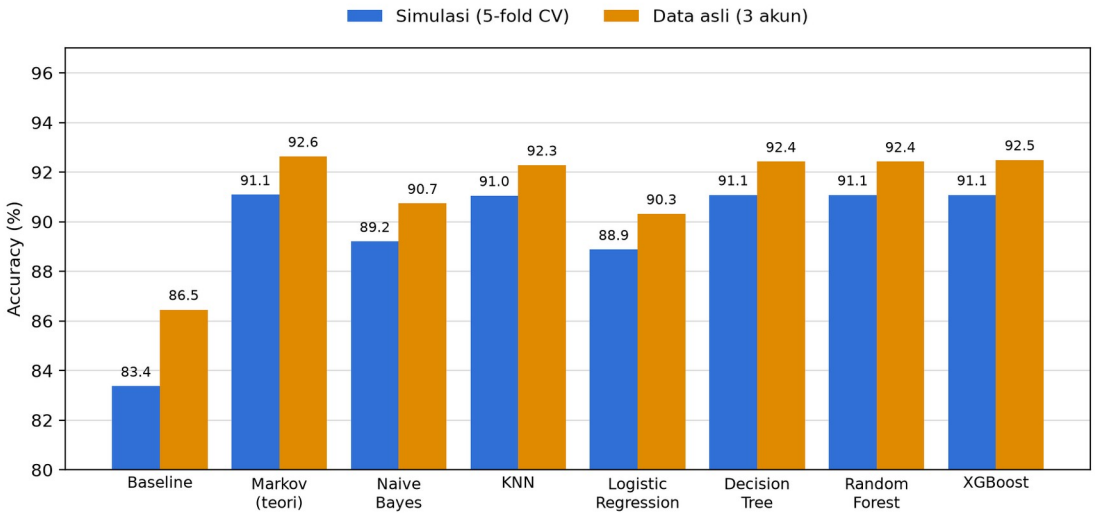
Semua model, termasuk baseline yang selalu menebak "tidak dapat S-Rank", memperoleh akurasi sekitar **98,3%**. Hal ini terjadi karena S-Rank sangat jarang (±1,7%). Dengan demikian, pada tugas A akurasi **tidak dapat membedakan** model yang baik dan model yang tidak berguna. Perbedaan baru terlihat pada ROC-AUC dan Log Loss, di mana model berbasis pohon paling dekat dengan Markov Chain.

4.5 Hasil Tugas B: S-Rank dalam 10 Pull

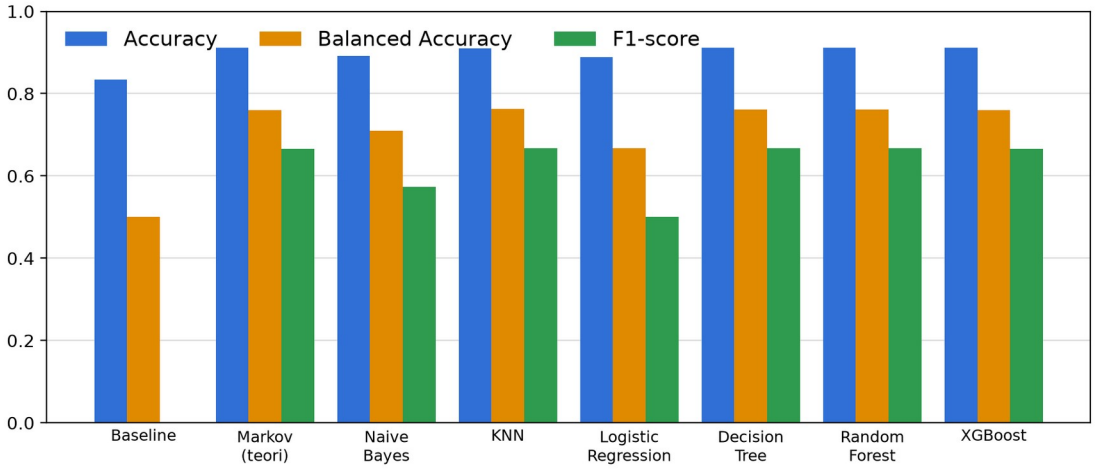
Tabel 4.4 Hasil cross-validation 5-fold tugas B (rata-rata) dan uji data asli

Model	Acc. simulasi	F1 simulasi	Log Loss simulasi	Acc. data asli	F1 data asli
Baseline (selalu "tidak S")	0,8337	0,0000	2,6804	0,8646	0,0000
Markov Chain (teori)	0,9108	0,6658	0,2811	0,9263	0,6465
Naive Bayes	0,8920	0,5736	0,3488	0,9074	0,5714
KNN	0,9105	**0,6680**	0,3033	0,9228	0,6233
Logistic Regression	0,8888	0,5010	0,3625	0,9032	0,4480
Decision Tree	**0,9108**	0,6671	**0,2814**	0,9242	0,6327
Random Forest	**0,9108**	0,6669	0,2816	0,9242	0,6351
XGBoost	**0,9108**	0,6661	0,2815	**0,9249**	**0,6373**

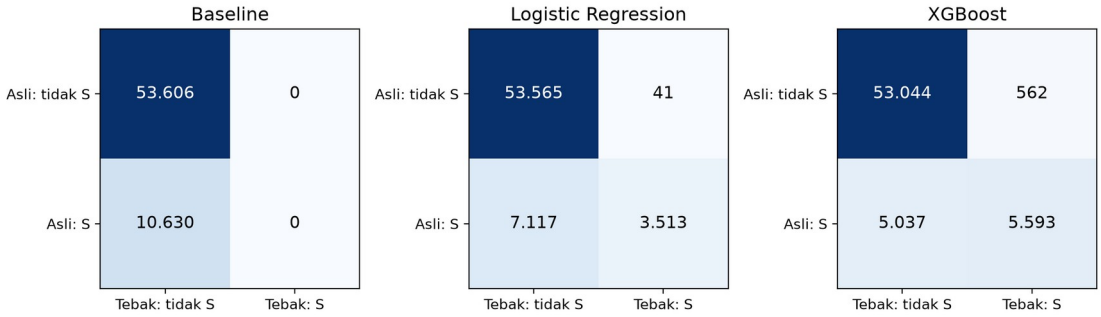
Angka tebal adalah nilai terbaik di antara keenam algoritma. Pada data simulasi, Decision Tree, Random Forest, dan XGBoost memperoleh akurasi 0,9108, diikuti KNN (0,9105). Menurut uji signifikansi (Subbab 4.7), keempatnya tidak berbeda nyata, sedangkan Naive Bayes (0,8920) dan Logistic Regression (0,8888) nyata lebih rendah. Akurasi pada data asli terlihat lebih tinggi (±92%) karena proporsi kelas positifnya lebih kecil (13,5% dibanding 16,6%), sehingga baseline pun naik dari 83,4% menjadi 86,5%. Dibandingkan dengan baseline masing-masing, keunggulan model terbaik adalah 7,7 poin di simulasi dan 6,0 poin di data asli.



Gambar 4.5 Perbandingan akurasi tugas B pada data simulasi dan data asli



Gambar 4.6 Accuracy, Balanced Accuracy, dan F1-score tugas B (cross-validation)



Gambar 4.7 Confusion matrix tugas B pada data uji simulasi

4.6 Hasil Latih dan Uji dengan Data Asli Saja

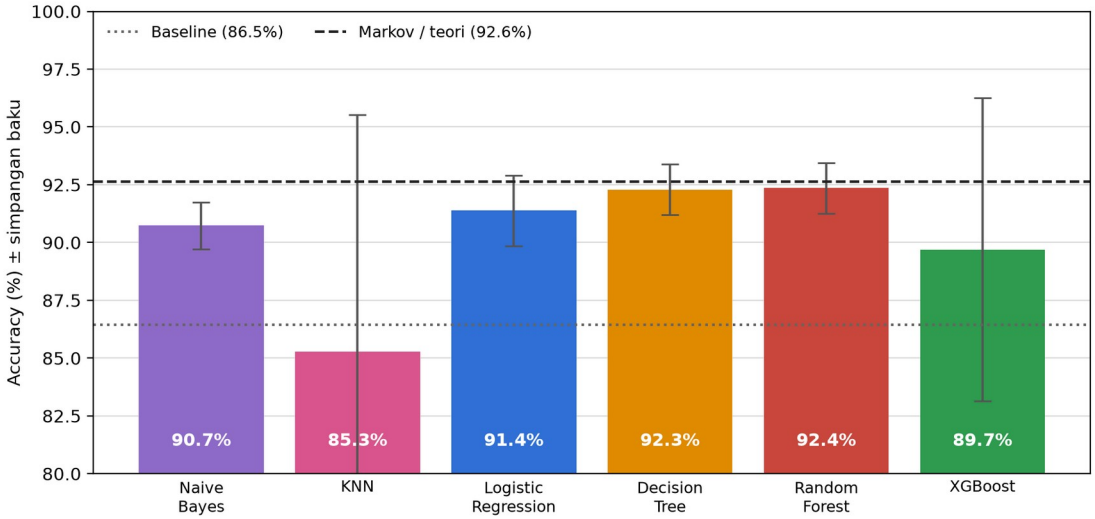
Pada eksperimen kedua, keenam algoritma dilatih dan diuji **hanya dengan data asli** (3 akun, 1.425 baris, 193 kelas positif pada tugas B). Data dibagi per siklus pity menjadi 5 bagian (GroupKFold 5-fold, 25 siklus), sehingga pull dari satu siklus tidak pernah berada di data latih dan data uji sekaligus. Hyperparameter dipilih ulang dengan grid search di dalam data latih setiap fold (*nested cross-validation*). Hasilnya ditunjukkan pada Tabel 4.5, Tabel 4.6, dan Gambar 4.8.

Tabel 4.5 Hasil latih–uji dengan data asli saja, tugas B (rata-rata 5-fold, nested grid search)

Model	Accuracy	Bal. Acc.	F1	ROC-AUC	Log Loss
Baseline (selalu "tidak S")	0,8645 ± 0,0119	0,5000	0,0000	0,5000	2,1835
Markov Chain (teori)	0,9264 ± 0,0149	0,7470	0,6389	0,7825	0,2527
Naive Bayes	0,9073 ± 0,0101	0,6765	0,5108	0,8455	**0,2847**
KNN	0,8529 ± 0,1024	0,6476	0,4412	0,7757	0,7433
Logistic Regression	0,9138 ± 0,0154	0,7105	0,5591	0,8391	0,2966
Decision Tree	0,9229 ± 0,0110	**0,7559**	**0,6440**	**0,8505**	0,4477
Random Forest	**0,9235 ± 0,0109**	0,7438	0,6309	0,8323	0,3904
XGBoost	0,8970 ± 0,0656	0,7395	0,5958	0,7900	0,3969

Tabel 4.6 Pengaruh grid search pada eksperimen data asli (accuracy rata-rata 5-fold)

Algoritma	Pengaturan awal	Nested grid search	Pengaturan terpilih di setiap fold
Naive Bayes	0,8654	0,9073	var_smoothing = 0,1 (4 fold); var_smoothing = 10^{-3} (1 fold)
KNN	0,8837	0,8529	k = 51 (4 fold); k = 15 (1 fold)
Logistic Regression	0,9201	0,9138	C = 0,1 (3 fold); C = 10 (1 fold); C = 1 (1 fold)
Decision Tree	0,9229	0,9229	max_depth = 6, min_samples_leaf = 50 (5 fold)
Random Forest	0,9221	0,9235	max_depth = 8, min_samples_leaf = 20 (2 fold); max_depth = 12, min_samples_leaf = 10 (1 fold); max_depth = 6, min_samples_leaf = 10 (1 fold); max_depth = 8, min_samples_leaf = 10 (1 fold)
XGBoost	0,8970	0,8970	learning_rate = 0,1, max_depth = 6 (3 fold); learning_rate = 0,1, max_depth = 4 (1 fold); learning_rate = 0,05, max_depth = 3 (1 fold)



Gambar 4.8 Akurasi enam algoritma yang dilatih dan diuji dengan data asli saja

- **Random Forest dan Decision Tree teratas:** Random Forest 92,35% dan Decision Tree 92,29%. Selisih Random Forest dengan acuan teori Markov Chain (92,64%) hanya 0,28 poin.
- **Grid search tidak selalu membantu.** Naive Bayes naik dari 86,54% menjadi 90,73%, tetapi KNN turun dari 88,37% menjadi 85,29% dan Logistic Regression turun dari 92,01% menjadi 91,38%. Pengaturan dipilih hanya dari data latih setiap fold; dengan data sekecil ini, pengaturan yang terbaik di data latih belum tentu terbaik di data uji.
- **KNN dan XGBoost tidak stabil.** Pada fold ke-4, akurasi KNN hanya 64,9% dan XGBoost hanya 77,1%, sedangkan Decision Tree dan Random Forest pada fold yang sama 92,1% dan 92,1%. Data latih fold tersebut hanya memuat satu siklus Agent berstatus *guaranteed*, yaitu siklus yang kebetulan mendapat S-Rank di pull ke-38. Dari satu siklus itu, KNN dan XGBoost mempelajari pola keliru "guaranteed berarti S-Rank segera keluar", lalu salah menebak "dapat S-Rank" pada dua siklus guaranteed lain di data uji yang ternyata panjang. Decision Tree dan Random Forest tidak terpengaruh karena aturan min_samples_leaf mencegah cabang yang hanya didukung sedikit data. Ini contoh *overfitting* pada fitur yang tidak relevan ketika data sedikit.
- **Model pohon hampir tidak berubah.** Pengaturan awal Decision Tree terpilih di 5 dari 5 fold, dan akurasi Random Forest serta XGBoost berubah paling banyak 0,14 poin.

- **Akurasi naik, recall turun pada Naive Bayes.** Grid search memilih var_smoothing yang besar sehingga Naive Bayes lebih jarang menebak "dapat S-Rank": akurasinya naik, tetapi recall turun dari 0,65 menjadi 0,36 dan Balanced Accuracy dari 0,7762 menjadi 0,6765. Hal ini terjadi karena grid search mengoptimalkan accuracy. Jika yang diutamakan adalah menangkap kelas minoritas, metrik pemilihan perlu diganti, misalnya dengan Balanced Accuracy.
- Simpangan baku (±1,0–10,2%) jauh lebih besar daripada pada data simulasi (±0,1–0,2%) karena jumlah data asli jauh lebih sedikit.

4.7 Uji Signifikansi Antar-Model

Tabel 4.7 menunjukkan hasil uji t berpasangan terkoreksi (Subbab 2.7) pada tugas B. Setiap model dibandingkan dengan algoritma berakurasi rata-rata tertinggi sebelum dibulatkan, yaitu XGBoost pada data simulasi dan Random Forest pada data asli. Selisih bernilai negatif berarti model tersebut lebih rendah daripada acuan, dan p-value bercetak tebal menandakan selisih yang signifikan ($p < 0,05$).

Tabel 4.7 Uji t berpasangan terkoreksi tugas B (5-fold)

Model	Selisih dari XGBoost (simulasi)	p-value	Selisih dari Random Forest (data asli)	p-value
Baseline (selalu "tidak S")	−7,71 poin	**< 0,001**	−5,90 poin	**0,003**
Markov Chain (teori)	0,00 poin	0,541	+0,28 poin	0,541
Naive Bayes	−1,88 poin	**< 0,001**	−1,62 poin	0,297
KNN	−0,03 poin	0,572	−7,06 poin	0,409
Logistic Regression	−2,20 poin	**< 0,001**	−0,98 poin	0,193
Decision Tree	0,00 poin	0,890	−0,07 poin	0,802
Random Forest	0,00 poin	0,735	acuan	–
XGBoost	acuan	–	−2,66 poin	0,601

- **Data simulasi:** KNN, Decision Tree dan Random Forest tidak berbeda nyata dari XGBoost ($p \geq 0,57$), begitu pula acuan teori Markov Chain ($p = 0,541$). Naive Bayes dan Logistic Regression nyata lebih rendah ($p < 0,001$). Jadi pada data yang besar, keunggulan model berbasis pohon dan KNN atas Naive Bayes dan Logistic Regression bukan kebetulan.

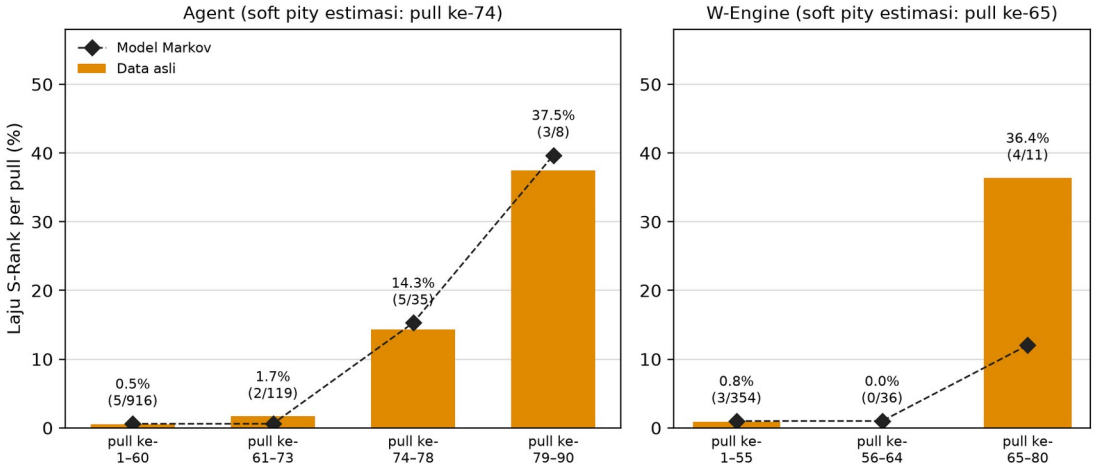
- **Data asli:** hanya baseline yang nyata lebih rendah daripada Random Forest (p = 0,003). Selisih Random Forest dengan kelima algoritma lain (0,07–7,06 poin) tidak signifikan (p = 0,19–0,80).
- Dengan 25 siklus pity yang dibagi menjadi 5 fold, data asli belum cukup untuk membuktikan bahwa satu algoritma benar-benar lebih unggul daripada yang lain. Yang dapat disimpulkan dengan yakin adalah Random Forest lebih baik daripada baseline, sedangkan urutan keenam algoritma pada data asli masih mungkin berubah jika data bertambah.

4.8 Pola Soft Pity pada Data Asli

Untuk menjawab rumusan masalah kedua, laju S-Rank per pull pada data asli dihitung untuk setiap rentang pull sejak S-Rank terakhir, lalu dibandingkan dengan rata-rata peluang model Markov pada rentang yang sama. Hasilnya ditunjukkan pada Tabel 4.8 dan Gambar 4.9.

Tabel 4.8 Laju S-Rank per rentang pull: data asli dan model Markov

Banner	Pull ke-	Jumlah pull	S-Rank	Laju data asli	Model Markov
Agent	1–60	916	5	0,5%	0,6%
Agent	61–73	119	2	1,7%	0,6%
Agent	74–78	35	5	**14,3%**	15,3%
Agent	79–90	8	3	**37,5%**	39,7%
W-Engine	1–55	354	3	0,8%	1,0%
W-Engine	56–64	36	0	0,0%	1,0%
W-Engine	65–80	11	4	**36,4%**	12,0%



Gambar 4.9 Pola soft pity pada data asli dibandingkan model Markov

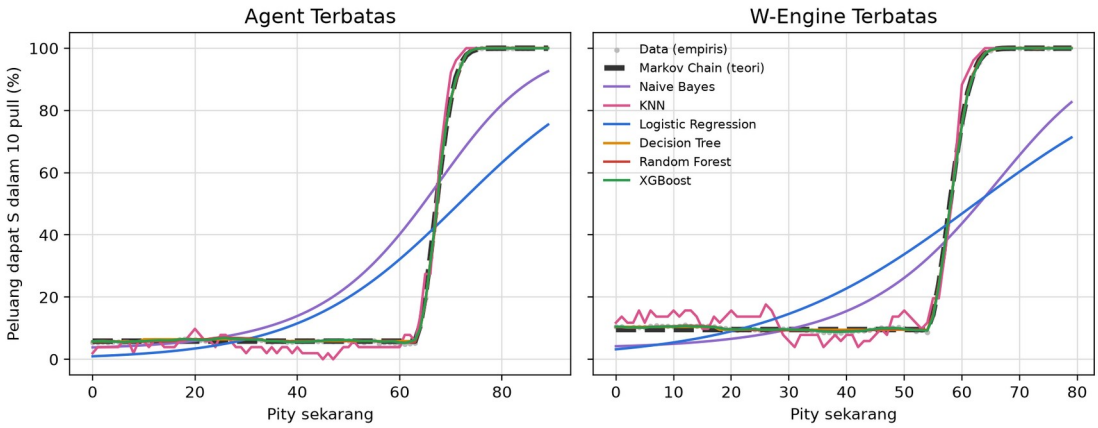
- **Agent:** laju S-Rank hampir nol sampai pull ke-73 (7 S-Rank dari 1.035 pull), lalu melonjak menjadi 14,3% pada pull ke-74–78 dan 37,5% pada pull ke-79–90. Pola ini sangat dekat dengan model Markov yang memakai estimasi soft pity pull ke-74.
- **W-Engine:** sebelum pull ke-65 hanya 3 S-Rank dari 390 pull (0,8%), lalu 4 dari 11 pull pada pull ke-65 ke atas menghasilkan S-Rank (36,4%), lebih tinggi daripada model (12,0%). Hal ini mengisyaratkan bahwa peluang W-Engine setelah soft pity mungkin naik lebih cepat daripada asumsi, tetapi datanya masih terlalu sedikit untuk disimpulkan.
- Decision Tree yang dilatih **hanya dengan data asli** (max_depth 6, min_samples_leaf 50, pengaturan yang terpilih di setiap fold nested CV) memisahkan data pertama kali (akar pohon) pada pity_ratio 0,724, yaitu pity ±65 untuk Agent dan ±58 untuk W-Engine. Titik ini sekitar 8–10 pull sebelum soft pity, sama seperti Decision Tree yang dilatih dengan data simulasi (pity_ratio 0,71–0,74).
- Dengan demikian, lonjakan peluang yang tidak diumumkan resmi **dapat ditemukan dari data asli**, baik melalui analisis laju S-Rank maupun oleh algoritma klasifikasi yang tidak diberi tahu tentang soft pity.

4.9 Pembahasan

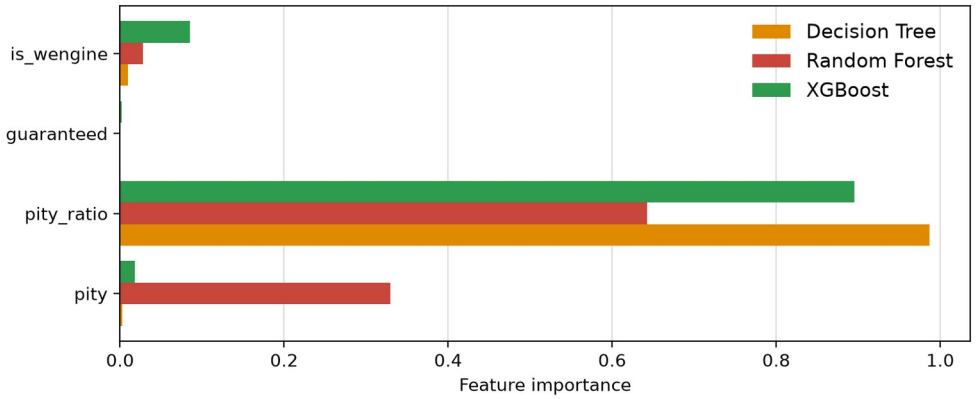
1. **Jebakan akurasi.** Pada tugas A, baseline sudah mencapai akurasi 98,28%. Akurasi saja tidak dapat dipakai untuk menilai model pada data yang sangat tidak seimbang, sehingga diperlukan Balanced Accuracy, F1-score, ROC-AUC, dan Log Loss.
2. **Algoritma terbaik.** Pada tugas B, Decision Tree, Random Forest, XGBoost, dan KNN mencapai akurasi ±91,1% di simulasi dan ±92,3–92,5% di data asli. Pada data simulasi, nilai ini sama dengan batas atas teori (Markov Chain) dan perbedaan di antara keempatnya tidak signifikan, sehingga keempat algoritma tersebut sudah bekerja maksimal.
3. **Model pohon menemukan soft pity sendiri.** Aturan Decision Tree memisahkan data di sekitar pity_ratio 0,71–0,74 pada data simulasi dan 0,724 pada data asli, yaitu sekitar 10 pull sebelum soft pity, tepat saat jendela 10 pull ke depan mulai menyentuh soft pity. Model tidak pernah diberi tahu tentang soft pity (Gambar 4.10).
4. **Logistic Regression dan Naive Bayes kalah**, dan selisihnya signifikan pada data simulasi walaupun hyperparameter keduanya sudah dipilih

dengan grid search. Logistic Regression hanya dapat membentuk kurva sigmoid yang halus sehingga tidak bisa meniru lonjakan tajam setelah soft pity. Naive Bayes mengasumsikan fitur saling bebas dan berdistribusi normal, padahal pity dan pity_ratio sangat berkaitan. Log Loss keduanya paling besar di antara keenam algoritma.

5. **KNN setara model pohon pada data simulasi** karena fiturnya sedikit dan nilainya berulang, sehingga tetangga terdekat praktis adalah pull dengan pity yang sama. Kelemahannya, KNN menyimpan seluruh data latih, paling lambat saat prediksi, dan tidak stabil pada data asli yang kecil (Subbab 4.6).
6. **Fitur guaranteed hampir tidak berpengaruh** pada model yang dilatih dengan data simulasi (Gambar 4.11). Hal ini sesuai aturan permainan: guaranteed hanya menentukan S-Rank mana yang keluar, bukan kapan S-Rank keluar. Namun pada data asli yang kecil, fitur ini justru dapat menyesatkan KNN dan XGBoost ketika siklus guaranteed di data latih hanya sedikit (Subbab 4.6).
7. **Konsisten pada data asli.** Urutan kinerja algoritma pada data asli sama dengan pada simulasi, dan laju S-Rank per pull pada data asli mengikuti model Markov (Subbab 4.8). Artinya, asumsi soft pity yang dipakai simulasi tidak bertentangan dengan perilaku gacha sungguhan, dengan catatan data asli baru berasal dari 3 akun.
8. **Tidak bergantung pada simulasi.** Ketika dilatih dan diuji dengan data asli saja (1.425 baris), Random Forest dan Decision Tree tetap tertinggi (92,3–92,4%), walaupun selisihnya dengan algoritma lain belum signifikan. Kesimpulan penelitian tidak berubah walaupun data simulasi tidak dipakai sama sekali.
9. **Pengaruh grid search.** Grid search paling berpengaruh pada Naive Bayes, yang pengaturan awalnya tidak cocok dengan data, sedangkan model berbasis pohon hampir tidak berubah. Pada data asli yang kecil, grid search di dalam setiap fold tidak selalu memilih pengaturan yang lebih baik, sehingga hasilnya perlu dibaca bersama simpangan bakunya. Kesimpulan tentang algoritma terbaik tidak berubah setelah grid search.
10. **Nilai tambah penambangan data.** Rumus resmi tidak memuat soft pity, sedangkan algoritma klasifikasi yang hanya diberi data mampu mencapai akurasi yang mendekati acuan teori dan menemukan titik lonjakannya sendiri. Hal inilah yang tidak dapat diperoleh dari rumus resmi saja.



Gambar 4.10 Kurva peluang mendapat S-Rank dalam 10 pull menurut setiap model



Gambar 4.11 Feature importance model berbasis pohon (tugas B)

BAB V
PENUTUP

5.1 Kesimpulan

1. Sistem pity Zenless Zone Zero dapat dimodelkan dengan Markov Chain. Dari parameter banner, aplikasi menghitung peluang mendapat S-Rank dan S-Rank rate-up dalam N pull secara langsung, dengan catatan titik soft pity masih berupa estimasi.
2. Pola soft pity yang tidak diumumkan resmi dapat ditemukan dari data asli. Laju S-Rank Agent melonjak mulai pull ke-74 (dari di bawah 1% menjadi 14,3%) dan W-Engine sekitar pull ke-65, dan Decision Tree yang dilatih hanya dengan data asli menemukan titik pemisah yang sesuai tanpa diberi tahu.
3. Untuk memprediksi S-Rank dalam 10 pull berikutnya, Decision Tree, Random Forest, XGBoost, dan KNN memberikan akurasi tertinggi dan tidak berbeda nyata satu sama lain (±91,1% pada simulasi, ±92,3–92,5% pada data asli), hampir menyamai acuan teori. Naive Bayes (89,2%) dan Logistic Regression (88,9%) nyata lebih rendah pada data simulasi walaupun hyperparameter-nya sudah dipilih dengan grid search. Saat dilatih dan diuji hanya dengan data asli (1.425 baris), Random Forest memperoleh akurasi tertinggi (92,35%), tetapi selisihnya dengan kelima algoritma lain belum signifikan karena data asli baru 25 siklus pity.
4. Grid search terutama memperbaiki Naive Bayes. Pada data asli yang kecil, hasil grid search tidak selalu lebih baik, dan algoritma yang fleksibel seperti KNN dapat mempelajari pola keliru dari fitur guaranteed; kesimpulan tentang algoritma terbaik tidak berubah.
5. Pada data yang sangat tidak seimbang, akurasi saja tidak cukup; Balanced Accuracy, F1-score, ROC-AUC, dan Log Loss diperlukan.
6. Aplikasi web berhasil dibangun dengan Streamlit dan dapat diakses dari ponsel maupun komputer di https://project-data-mining-zzz.streamlit.app.

5.2 Keterbatasan

- Data latih utama berasal dari simulasi yang dibangkitkan dari estimasi soft pity, sehingga model yang dilatih dengan simulasi mempelajari ulang asumsi tersebut. Keterbatasan ini dikurangi dengan eksperimen dan analisis yang hanya memakai data asli.

- Data asli baru berasal dari 3 akun (28 S-Rank, 25 siklus pity). Baris-baris dalam satu siklus saling berkaitan, sehingga informasi yang sebenarnya lebih sedikit daripada jumlah baris, dan simpangan baku hasilnya masih ±1,0–10,2%. Akibatnya, uji signifikansi pada data asli memiliki daya uji yang rendah: selisih kecil antaralgoritma sulit dibuktikan walaupun mungkin ada.
- Siklus berstatus guaranteed pada data asli masih sangat sedikit, sehingga algoritma yang fleksibel seperti KNN dan XGBoost dapat mempelajari pola keliru dari fitur guaranteed (Subbab 4.6).
- Grid search hanya mencoba ruang pencarian yang terbatas (Tabel 3.3) dan memakai accuracy sebagai metrik pemilihan, sehingga pengaturan yang terpilih belum tentu terbaik untuk metrik lain seperti Balanced Accuracy.
- Penentuan menang/kalah 50/50 bergantung pada daftar S-Rank standar yang harus diperbarui jika isi banner standar berubah, dan server hanya menyimpan riwayat beberapa bulan terakhir.

5.3 Saran

- Menambah jumlah akun data asli (minimal 5 akun, idealnya 10 akun atau lebih) agar eksperimen latih–uji dengan data asli lebih stabil dan uji signifikansi memiliki daya uji yang cukup.
- Memilih hyperparameter dengan metrik lain (misalnya Balanced Accuracy atau Log Loss) sesuai kebutuhan, lalu membandingkan hasilnya.
- Mengestimasi titik dan kenaikan soft pity, khususnya pada banner W-Engine, dari data asli yang lebih banyak sebagai pengganti estimasi komunitas.
- Memperluas cakupan ke banner standar, banner Bangboo, dan item A-Rank.
- Menyediakan cara impor riwayat langsung dari ponsel Android.

DAFTAR PUSTAKA

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*(1), 21–27.

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*(9), 1263–1284.

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley.

HoYoverse. (2024). *Zenless Zone Zero* [Permainan video]. Menu detail Signal Search: peluang dan aturan banner.

Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning, 52*(3), 239–281.

Norris, J. R. (1997). *Markov chains*. Cambridge University Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*(1), 81–106.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), e0118432.

Streamlit Inc. (2024). *Streamlit documentation*. https://docs.streamlit.io

UIGF Organization. (2024). *Uniformed Interchangeable GachaLog Format standard v4*. https://uigf.org

Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics, 7*, 91.

LAMPIRAN

Tautan aplikasi: https://project-data-mining-zzz.streamlit.app

Kode sumber: https://github.com/tehgeii/project-data-mining

Video presentasi (YouTube): [isi tautan video masing-masing anggota]

Cara menjalankan secara lokal: pasang Python 3.12, lalu jalankan perintah pip install -r requirements-dev.txt, lalu streamlit run streamlit_app.py; pengujian dijalankan dengan python -m pytest -q.